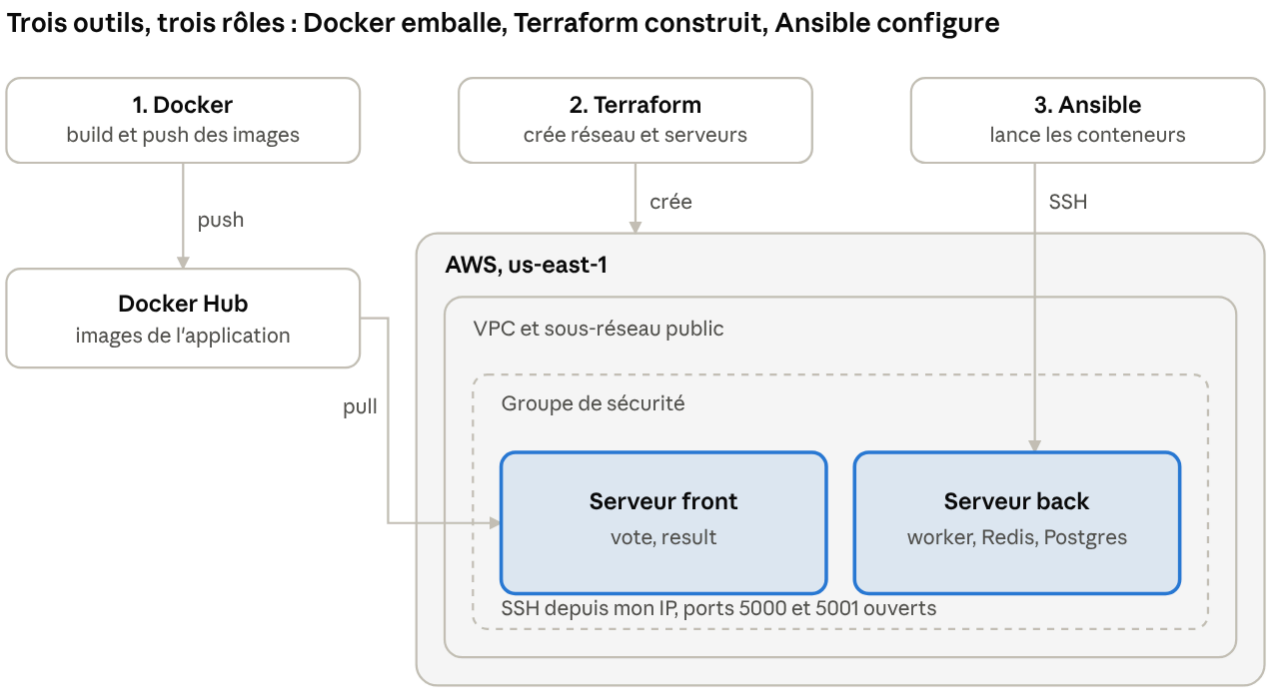
Guide pas à pas : déployer une application avec Docker, Terraform et Ansible sur AWS

Oct 27, 2026 · Dally NTOLO

Vue d'ensemble et prérequis

Ce guide déploie une application multi-services (par exemple une appli de vote : interface de vote, page de résultats, worker, Redis, Postgres) sur des serveurs AWS, avec trois outils qui se complètent :

1. **Docker** : chaque service est empaqueté dans une image publiée sur Docker Hub.
2. **Terraform** : crée l'infrastructure AWS (réseau, pare-feu, serveurs EC2).
3. **Ansible** : se connecte aux serveurs, installe Docker et lance les conteneurs.



vue d'ensemble, exemple à deux serveurs

Exemple à deux serveurs : Docker publie les images, Terraform crée les machines, Ansible s'y connecte et lance les conteneurs, qui téléchargent leurs images depuis Docker Hub.

Les noms de services, ports et répartition sur les serveurs sont des exemples à adapter à l'application.

Outils à installer sur le poste : AWS CLI v2 (configuré avec `aws configure`), Terraform, Ansible, Docker, git.

Il faut aussi : un compte AWS, un compte Docker Hub et une paire de clés SSH.

```
aws --version && terraform version && ansible --version && docker --version
aws sts get-caller-identity
```

Variables à adapter

On définit les valeurs une fois, les commandes suivantes les réutilisent.

```
export AWS_REGION=us-east-1
export PREFIX=mon-projet                # préfixe distinctif pour toutes les
ressources
export DH_USER=mon-compte-dockerhub
export KEY_NAME=${PREFIX}-key
export MY_IP=$(curl -s https://checkip.amazonaws.com)/32
```

Créer la paire de clés SSH et l'enregistrer dans AWS :

```
ssh-keygen -t ed25519 -f ~/.ssh/${KEY_NAME} -N ""
aws ec2 import-key-pair --key-name $KEY_NAME \
  --public-key-material fileb://~/.ssh/${KEY_NAME}.pub --region $AWS_REGION
```

La clé privée ne doit jamais être envoyée dans Git ni partagée.

Étape 1 : conteneuriser avec Docker et tester en local

Chaque service a son `Dockerfile`. On construit, on teste avec Docker Compose, puis on publie sur Docker Hub.

```
docker login -u $DH_USER

docker build -t $DH_USER/vote:1.0   ./vote
docker build -t $DH_USER/result:1.0 ./result
docker build -t $DH_USER/worker:1.0 ./worker
```

Test local avant toute chose (`docker-compose.yml` à la racine) :

```
docker compose up -d
docker compose ps
curl -I http://localhost:5000     # interface de vote
curl -I http://localhost:5001     # résultats
docker compose down -v
```

Publication :

```
docker push $DH_USER/vote:1.0
docker push $DH_USER/result:1.0
docker push $DH_USER/worker:1.0
```

Critère de réussite : l'application fonctionne en local et les images sont visibles sur Docker Hub. Ne pas passer à l'étape suivante tant que ce test local échoue.

Étape 2 : créer l'infrastructure AWS avec Terraform

Terraform décrit le réseau, le pare-feu et les serveurs dans des fichiers. Dossier `terraform/`, fichier `main.tf` :

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

variable "region"   { default = "us-east-1" }
variable "prefix"   { default = "mon-projet" }
variable "key_name" {}
variable "my_ip"    {}

provider "aws" {
  region = var.region
  default_tags { tags = { project = var.prefix } }
}

data "aws_ssm_parameter" "ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-
x86_64"
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "${var.prefix}-vpc" }
}

resource "aws_internet_gateway" "igw" { vpc_id = aws_vpc.main.id }

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
```

```
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "srv" {
  name   = "${var.prefix}-sg"
  vpc_id = aws_vpc.main.id

  ingress {            # SSH uniquement depuis mon adresse
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }
  ingress {            # interfaces web
    from_port   = 5000
    to_port     = 5001
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {            # communication entre les serveurs du réseau
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    self      = true
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

locals { servers = ["front", "back"] }   # adapter à l'application
```

```
resource "aws_instance" "srv" {
  for_each               = toset(local.servers)
  ami                    = data.aws_ssm_parameter.ami.value
  instance_type          = "t3.small"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.srv.id]
  key_name               = var.key_name
  tags = { Name = "${var.prefix}-${each.key}" }
}

# Génère l'inventaire utilisé par Ansible
resource "local_file" "inventory" {
  filename = "${path.module}/../ansible/inventory.ini"
  content  = join("\n", concat(
    ["[servers]"],
    [for k, i in aws_instance.srv : "${k} ansible_host=${i.public_ip}
private_ip=${i.private_ip}"],
    ["", "[servers:vars]", "ansible_user=ec2-user"]
  ))
}

output "public_ips" { value = { for k, i in aws_instance.srv : k =>
i.public_ip } }
```

Commandes, toujours dans cet ordre :

```
cd terraform
terraform init
terraform fmt && terraform validate
terraform plan -var key_name=$KEY_NAME -var my_ip=$MY_IP -var prefix=$PREFIX
terraform apply -var key_name=$KEY_NAME -var my_ip=$MY_IP -var prefix=$PREFIX
terraform output
```

Lire le `plan` avant chaque `apply` : il indique exactement ce qui sera créé, modifié ou détruit. Ajouter `terraform.tfstate` et le dossier `.terraform/` au `.gitignore`.

Étape 3 : configurer les serveurs avec Ansible

Terraform a généré `ansible/inventory.ini`. Ansible installe Docker puis lance les conteneurs de chaque serveur.

```
cd ../ansible
ansible-galaxy collection install community.docker

cat > ansible.cfg <<EOF
[defaults]
inventory = inventory.ini
private_key_file = ~/.ssh/${KEY_NAME}
host_key_checking = False
EOF

ansible all -m ping          # doit répondre "pong" pour chaque serveur
```

Si le test échoue juste après la création des serveurs, attendre une minute : ils démarrent encore.

Contenu de chaque serveur, un fichier par serveur (`host_vars/front.yml` , `host_vars/back.yml`) :

```
# host_vars/front.yml
containers:
  - { name: vote,   image: "mon-compte-dockerhub/vote:1.0",   ports:
["5000:80"] }
  - { name: result, image: "mon-compte-dockerhub/result:1.0", ports:
["5001:80"] }
```

Playbook (`site.yml`) :

```
- hosts: servers
  become: true
  tasks:
    - name: Installer Docker et pip
      ansible.builtin.dnf:
        name: [docker, python3-pip]
        state: present

    - name: Démarrer Docker
      ansible.builtin.service:
        name: docker
        state: started
        enabled: true

    - name: Installer le SDK Python de Docker
      ansible.builtin.pip:
        name: docker

    - name: Lancer les conteneurs
```

```
      community.docker.docker_container:
        name: "{{ item.name }}"
        image: "{{ item.image }}"
        ports: "{{ item.ports | default([]) }}"
        env: "{{ item.env | default({}) }}"
        restart_policy: always
        state: started
      loop: "{{ containers | default([]) }}"
```

```
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml
```

Le playbook est idempotent : le relancer ne change que ce qui diffère. Les mots de passe de la base ne se mettent pas en clair dans le dépôt : utiliser `ansible-vault`.

Étape 4 : déployer et vérifier

```
cd ../terraform && terraform output public_ips

curl -I http://<ip-publique-front>:5000     # interface de vote, doit
répondre 200
curl -I http://<ip-publique-front>:5001     # résultats

ansible servers -a "docker ps"               # conteneurs en cours sur chaque
serveur
ansible front -a "docker logs vote --tail 20"
```

Faire un vote depuis le navigateur et vérifier qu'il apparaît sur la page de résultats : cela prouve que toute la chaîne (vote, file d'attente, worker, base) fonctionne.

Pour mettre à jour l'application : publier une nouvelle version de l'image, changer le tag dans `host_vars/`, puis relancer `ansible-playbook site.yml`.

Étape 5 : nettoyage complet

Terraform supprime tout ce qu'il a créé en une commande, donc rien n'est oublié et rien n'est facturé ensuite.

```
cd terraform
terraform plan -destroy -var key_name=$KEY_NAME -var my_ip=$MY_IP -var
prefix=$PREFIX
terraform destroy -var key_name=$KEY_NAME -var my_ip=$MY_IP -var
prefix=$PREFIX

# La paire de clés a été créée hors Terraform : la supprimer séparément
aws ec2 delete-key-pair --key-name $KEY_NAME --region $AWS_REGION
```

Vérifier ensuite dans la console AWS qu'il ne reste aucune instance EC2 ni aucun VPC portant le préfixe du projet. Dans un compte partagé, ne supprimer que les ressources qui portent ce préfixe.

Annexe : variantes avec les technologies non utilisées

Le guide combine Docker Hub, Terraform et Ansible sur des serveurs EC2. Voici les alternatives possibles et ce qu'elles changent.

Choix du guide	Alternative	Ce qui change
Ansible pour configurer les serveurs	`user_data` (cloud-init)	Docker s'installe au premier démarrage, sans outil supplémentaire, mais pas de mise à jour ensuite
Conteneurs lancés un par un	Docker Compose sur chaque serveur	Un seul fichier décrit tous les services d'un serveur
Docker Hub	Amazon ECR	Registre privé dans AWS, accès géré par IAM
Redis et Postgres en conteneurs	Amazon ElastiCache et RDS	Sauvegardes et mises à jour gérées par AWS, mais coût plus élevé
Serveurs EC2	Amazon ECS Fargate ou EKS	Plus de serveurs à gérer ; EKS est l'objet du projet 2
Terraform	CloudFormation ou AWS CDK	Outils propres à AWS, sans fichier d'état à gérer
Image de serveur standard	Packer	Une image pré-installée avec Docker, démarrage plus rapide
État Terraform local	État distant dans S3	Indispensable pour travailler à plusieurs

Docker Compose à la place des conteneurs individuels. Un fichier `docker-compose.yml` par serveur, copié puis lancé par Ansible :

```
- name: Copier le fichier Compose
  ansible.builtin.copy:
    src: files/docker-compose.{{ inventory_hostname }}.yml
    dest: /home/ec2-user/docker-compose.yml

- name: Lancer les services
  community.docker.docker_compose_v2:
    project_src: /home/ec2-user
    state: present
```

`user_data` à la place d'Ansible. Dans la ressource `aws_instance` de Terraform :

```
  user_data = <<-EOF
    #!/bin/bash
    dnf install -y docker
    systemctl enable --now docker
    docker run -d --restart always -p 5000:80 mon-compte-dockerhub/vote:1.0
  EOF
```

État Terraform distant dans S3. Créer d'abord le bucket, puis déclarer :

```
terraform {
  backend "s3" {
    bucket = "<nom-du-bucket-d-etat>"
    key    = "projet-1/terraform.tfstate"
    region = "us-east-1"
    use_lockfile = true
  }
}
```

Puis `terraform init -migrate-state`. L'option de verrouillage `use_lockfile` dépend de la version de Terraform : vérifier sa documentation.